

LAB PROGRAM

CHAPTER 5

1. There are $3n$ piles of coins of varying size, you and your friends will take piles of coins as follows: In each step, you will choose any 3 piles of coins (not necessarily consecutive). Of your choice, Alice will pick the pile with the maximum number of coins. You will pick the next pile with the maximum number of coins. Your friend Bob will pick the last pile. Repeat until there are no more piles of coins. Given an array of integers `piles` where `piles[i]` is the number of coins in the i th pile. Return the maximum number of coins that you can have.

CODE

```
def maxCoins(piles):
    piles.sort()
    n=len(piles)
    left=0
    right=n-1
    result=0
    for _ in range(n//3):
        right-=1
        result+=piles[right]
        right-=1
        left+=1
    return result
piles=[2,4,1,2,7,8]
print("Example 1 Output:",maxCoins(piles))
piles=[2,4,5]
print("Example 2 Output:",maxCoins(piles))
```

OUTPUT

```
... Example 1 Output: 9
    Example 2 Output: 4
```

2. You are given a 0-indexed integer array `coins`, representing the values of the coins available, and an integer `target`. An integer x is obtainable if there exists a subsequence of coins that sums to x . Return the minimum number of coins of any value that need to be added to the array so that every integer in the range $[1, \text{target}]$ is obtainable. A subsequence of an array is a new non-empty array that is formed from the original array by deleting some (possibly none) of the elements without disturbing the relative positions of the remaining elements.

CODE

```
def minPatches(coins,target):
    coins.sort()
    reachable=1
    i=0
    patches=0
    while reachable<=target:
        if i<len(coins) and coins[i]<=reachable:
            reachable+=coins[i]
            i+=1
        else:
            reachable+=reachable
            patches+=1
    return patches
coins=[1,4,10]
target=19
print("Example 1 Output:",minPatches(coins,target))
coins=[1,4,10,5,7,19]
target=19
print("Example 2 Output:",minPatches(coins,target))
```

OUTPUT

```
... Example 1 Output: 2
    Example 2 Output: 1
```

3. You are given an integer array jobs, where jobs[i] is the amount of time it takes to complete the ith job. There are k workers that you can assign jobs to. Each job should be assigned to exactly one worker. The working time of a worker is the sum of the time it takes to complete all jobs assigned to them. Your goal is to devise an optimal assignment such that the maximum working time of any worker is minimized. Return the minimum possible maximum working time of any assignment.

CODE

```
def minimumTimeRequired(jobs,k):
    jobs.sort(reverse=True)
    workers=[0]*k
    def backtrack(index,limit):
```

```

    if index==len(jobs):
        return True
    job=jobs[index]
    used=set()
    for i in range(k):
        if workers[i] in used:
            continue
        if workers[i]+job<=limit:
            used.add(workers[i])
            workers[i]+=job
            if backtrack(index+1,limit):
                return True
            workers[i]-=job
        if workers[i]==0:
            break
    return False
low=max(jobs)
high=sum(jobs)
while low<high:
    mid=(low+high)//2
    if backtrack(0,mid):
        high=mid
    else:
        low=mid+1
return low
jobs=[3,2,3]
k=3
print("Example 1 Output:",minimumTimeRequired(jobs,k))
jobs=[1,2,4,7,8]
k=2
print("Example 2 Output:",minimumTimeRequired(jobs,k))

```

OUTPUT

```
... Example 1 Output: 5
    Example 2 Output: 15
```

4. We have n jobs, where every job is scheduled to be done from $startTime[i]$ to $endTime[i]$, obtaining a profit of $profit[i]$. You're given the $startTime$, $endTime$ and $profit$ arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range. If you choose a job that ends at time X you will be able to start another job that starts at time X .

CODE

```
from bisect import bisect_right

def jobScheduling(startTime,endTime,profit):
    jobs=sorted(zip(startTime,endTime,profit))
    n=len(jobs)
    dp=[0]*(n+1)
    ends=[job[1] for job in jobs]
    for i in range(1,n+1):
        start,end,p=jobs[i-1]
        previous=bisect_right(ends,start,0,i-1)
        take=p+dp[previous]
        skip=dp[i-1]
        dp[i]=max(take,skip)
    return dp[n]

startTime=[1,2,3,3]
endTime=[3,4,5,6]
profit=[50,10,40,70]
print("Example 1 Output:",jobScheduling(startTime,endTime,profit))

startTime=[1,2,3,4,6]
endTime=[3,5,10,6,9]
profit=[20,20,100,70,60]
print("Example 2 Output:",jobScheduling(startTime,endTime,profit))
```

OUTPUT

```
... Example 1 Output: 120
    Example 2 Output: 120
```

5. Given a graph represented by an adjacency matrix, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to all other vertices in the graph. The graph is represented as an adjacency matrix where $graph[i][j]$ denote the weight of the edge from vertex i to vertex j . If there is no edge between vertices i and j , the value is Infinity (or a very large number).

CODE

```
INF=float('inf')
def dijkstra(graph,source):
    n=len(graph)
    dist=[INF]*n
    visited=[False]*n
    dist[source]=0
    for _ in range(n):
        u=-1
        min_dist=INF
        for i in range(n):
            if not visited[i] and dist[i]<min_dist:
                min_dist=dist[i]
                u=i
        if u==-1:
            break
        visited[u]=True
        for v in range(n):
            if not visited[v] and graph[u][v]!=INF:
                new_dist=dist[u]+graph[u][v]
                if new_dist<dist[v]:
                    dist[v]=new_dist
    return dist
graph=[
    [0,10,3,INF,INF],
    [INF,0,1,2,INF],
    [INF,4,0,8,2],
    [INF,INF,INF,0,7],
```

```

[INF,INF,INF,9,0]
]
source=0
print("Test Case 1 Output:",dijkstra(graph,source))

```

OUTPUT

```

└ ... Test Case 1 Output: [0, 7, 3, 9, 5]

```

6. Given a graph represented by an edge list, implement Dijkstra's Algorithm to find the shortest path from a given source vertex to a target vertex. The graph is represented as a list of edges where each edge is a tuple (u, v, w) representing an edge from vertex u to vertex v with weight w.

CODE

```

import heapq

def dijkstra(n,edges,source,target):
    graph=[[] for _ in range(n)]
    for u,v,w in edges:
        graph[u].append((v,w))
        graph[v].append((u,w))
    dist=[float('inf')]*n
    dist[source]=0
    pq=[(0,source)]
    while pq:
        d,node=heapq.heappop(pq)
        if d>dist[node]:
            continue
        for neighbor,weight in graph[node]:
            nd=d+weight
            if nd<dist[neighbor]:
                dist[neighbor]=nd
                heapq.heappush(pq,(nd,neighbor))
    return dist[target]

n=6
edges=[(0,1,7),(0,2,9),(0,5,14),(1,2,10),(1,3,15),(2,3,11),(2,5,2),(3,4,6),(4,5,9)]
source=0

```

```
target=4
```

```
print("Output:",dijkstra(n,edges,source,target))
```

OUTPUT

```
| ... Output: 20
```

7. Given a set of characters and their corresponding frequencies, construct the Huffman Tree and generate the Huffman Codes for each character.

CODE

```
import heapq
```

```
class Node:
```

```
    def __init__(self,char,freq):
```

```
        self.char=char
```

```
        self.freq=freq
```

```
        self.left=None
```

```
        self.right=None
```

```
    def __lt__(self,other):
```

```
        return self.freq<other.freq
```

```
def generateCodes(root,code,codes):
```

```
    if root is None:
```

```
        return
```

```
    if root.char is not None:
```

```
        codes[root.char]=code
```

```
    generateCodes(root.left,code+"0",codes)
```

```
    generateCodes(root.right,code+"1",codes)
```

```
def huffman(chars,freq):
```

```
    heap=[]
```

```
    for i in range(len(chars)):
```

```
        heapq.heappush(heap,Node(chars[i],freq[i]))
```

```
    while len(heap)>1:
```

```
        left=heapq.heappop(heap)
```

```
        right=heapq.heappop(heap)
```

```
        new=Node(None,left.freq+right.freq)
```

```
        new.left=left
```

```

        new.right=right
        heapq.heappush(heap,new)
    codes={}
    generateCodes(heap[0],"",codes)
    return codes
characters=['a','b','c','d']
frequencies=[5,9,12,13]
codes=huffman(characters,frequencies)
for ch in characters:
    print(ch,codes[ch])

```

OUTPUT

```

...  a 00
      b 01
      c 10
      d 11

```

8. Given a Huffman Tree and a Huffman encoded string, decode the string to get the original message.

CODE

```

import heapq
class Node:
    def __init__(self,char,freq):
        self.char=char
        self.freq=freq
        self.left=None
        self.right=None
    def __lt__(self,other):
        return self.freq<other.freq
def buildTree(chars,freq):
    heap=[]
    for i in range(len(chars)):
        heapq.heappush(heap,Node(chars[i],freq[i]))
    while len(heap)>1:
        left=heapq.heappop(heap)

```

```

    right=heapq.heappop(heap)
    new=Node(None,left.freq+right.freq)
    new.left=left
    new.right=right
    heapq.heappush(heap,new)
return heap[0]
def decode(root,encoded):
    result=""
    current=root
    for bit in encoded:
        if bit=="0":
            current=current.left
        else:
            current=current.right
        if current.char is not None:
            result+=current.char
            current=root
    return result
characters=['a','b','c','d']
frequencies=[5,9,12,13]
encoded_string="1101100111110"
root=buildTree(characters,frequencies)
print("Decoded string:",decode(root,encoded_string))

```

OUTPUT

```

... Decoded string: dcbbdd

```

9. Given a list of item weights and the maximum capacity of a container, determine the maximum weight that can be loaded into the container using a greedy approach. The greedy approach should prioritize loading heavier items first until the container reaches its capacity.

CODE

```

def maxWeight(weights,capacity):
    weights.sort(reverse=True)
    total=0

```

```

for weight in weights:
    if total+weight<=capacity:
        total+=weight
return total
weights=[10,20,30,40,50]
capacity=60
print("Test Case 1 Output:",maxWeight(weights,capacity))
weights=[5,10,15,20,25,30]
capacity=50
print("Test Case 2 Output:",maxWeight(weights,capacity))

```

OUTPUT

```

| ... Test Case 1 Output: 60
|   Test Case 2 Output: 50

```

10. Given a list of item weights and a maximum capacity for each container, determine the minimum number of containers required to load all items using a greedy approach. The greedy approach should prioritize loading items into the current container until it is full before moving to the next container.

CODE

```

def minimumContainers(weights,capacity):
    containers=1
    current=0
    for weight in weights:
        if current+weight<=capacity:
            current+=weight
        else:
            containers+=1
            current=weight
    return containers
weights=[5,10,15,20,25,30,35]
capacity=50
print("Test Case 1 Output:",minimumContainers(weights,capacity))
weights=[10,20,30,40,50,60,70,80]
capacity=100

```

```
print("Test Case 2 Output:",minimumContainers(weights,capacity))
```

OUTPUT

```
*** Test Case 1 Output: 4
    Test Case 2 Output: 5
```

11. Given a graph represented by an edge list, implement Kruskal's Algorithm to find the Minimum Spanning Tree (MST) and its total weight.

CODE

```
class DSU:
    def __init__(self,n):
        self.parent=list(range(n))
        self.rank=[0]*n
    def find(self,x):
        if self.parent[x]!=x:
            self.parent[x]=self.find(self.parent[x])
        return self.parent[x]
    def union(self,a,b):
        a=self.find(a)
        b=self.find(b)
        if a==b:
            return False
        if self.rank[a]<self.rank[b]:
            a,b=b,a
        self.parent[b]=a
        if self.rank[a]==self.rank[b]:
            self.rank[a]+=1
        return True
def kruskal(n,edges):
    edges=sorted(edges,key=lambda x:x[2])
    dsu=DSU(n)
    mst=[]
    total=0
    for u,v,w in edges:
        if dsu.union(u,v):
```

```

        mst.append((u,v,w))
        total+=w
        if len(mst)==n-1:
            break
    return mst,total

n=4
edges=[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)]
mst,total=kruskal(n,edges)
print("Test Case 1")
print("Edges in MST:",mst)
print("Total weight of MST:",total)

n=5
edges=[(0,1,2),(0,3,6),(1,2,3),(1,3,8),(1,4,5),(2,4,7),(3,4,9)]
mst,total=kruskal(n,edges)
print("\nTest Case 2")
print("Edges in MST:",mst)
print("Total weight of MST:",total)

```

OUTPUT

```

... Test Case 1
    Edges in MST: [(2, 3, 4), (0, 3, 5), (0, 1, 10)]
    Total weight of MST: 19

    Test Case 2
    Edges in MST: [(0, 1, 2), (1, 2, 3), (1, 4, 5), (0, 3, 6)]
    Total weight of MST: 16

```

12. Given a graph with weights and a potential Minimum Spanning Tree (MST), verify if the given MST is unique. If it is not unique, provide another possible MST.

CODE

```

from itertools import combinations

def spanningTree(n,edges,selected):
    parent=list(range(n))
    def find(x):
        while parent[x]!=x:
            parent[x]=parent[parent[x]]
        x=parent[x]

```

```

    return x
for i in selected:
    u,v,w=edges[i]
    a=find(u)
    b=find(v)
    if a==b:
        return False
    parent[a]=b
return True

def checkUniqueMST(n,edges,given_mst):
    all_msts=[]
    min_weight=float('inf')
    for selected in combinations(range(len(edges)),n-1):
        if spanningTree(n,edges,selected):
            weight=sum(edges[i][2] for i in selected)
            if weight<min_weight:
                min_weight=weight
                all_msts=[selected]
            elif weight==min_weight:
                all_msts.append(selected)
    given_weight=sum(w for u,v,w in given_mst)
    given_set=set(given_mst)
    possible_msts=[]
    for selected in all_msts:
        tree=[edges[i] for i in selected]
        possible_msts.append(tree)
    unique=len(all_msts)==1
    print("Is the given MST unique?",unique)
    print("Total weight of MST:",min_weight)
    if not unique:
        for tree in possible_msts:
            if set(tree)!=given_set:

```

```

        print("Another possible MST:",tree)
        break

n=4
edges=[(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)]
given_mst=[(2,3,4),(0,3,5),(0,1,10)]
print("Test Case 1")
checkUniqueMST(n,edges,given_mst)

n=5
edges=[(0,1,1),(0,2,1),(1,3,2),(2,3,2),(3,4,3),(4,2,3)]
given_mst=[(0,1,1),(0,2,1),(1,3,2),(3,4,3)]
print("\nTest Case 2")
checkUniqueMST(n,edges,given_mst)

```

OUTPUT

```

... Test Case 1
    Is the given MST unique? True
    Total weight of MST: 19

    Test Case 2
    Is the given MST unique? False
    Total weight of MST: 7
    Another possible MST: [(0, 1, 1), (0, 2, 1), (1, 3, 2), (4, 2, 3)]

```